

Τεχνητή Νοημοσύνη – Εργασία 2

Αλγόριθμοι Τοπικής Αναζήτησης

ΑΡΙΣΤΕΙΔΗΣ ΠΙΤΣΟΥΛΗΣ – 2022030024

ΒΑΣΙΛΕΙΟΣ ΚΑΡΑΒΑΝΑΣ – 2021030132

1 Εισαγωγή

Στην παρούσα εργασία υλοποιούμε τρεις αλγορίθμους τοπικής αναζήτησης για δύο κλασικά προβλήματα ελέγχου: *CartPole* και *MountainCar*. Σε αντίθεση με το τυπικό πλαίσιο, τα περιβάλλοντα εδώ είναι **μη-στατικά** (non-stationary): οι φυσικές τους παράμετροι αλλάζουν κατά τη διάρκεια της εκτέλεσης, οπότε δεν μπορούμε να βασιστούμε σε ένα σταθερό μοντέλο μετάβασης.

Για την προσομοίωση χρησιμοποιούμε τη βιβλιοθήκη *ns-gym*, η οποία επεκτείνει το *gymnasium* ώστε να υποστηρίζει δυναμικές αλλαγές παραμέτρων. Συγκεκριμένα:

- **CartPole**: η μάζα της ράβδου (*masspole*) αυξάνεται σταδιακά μέσω *IncrementUpdate*, ενώ η βαρύτητα (*gravity*) μεταβάλλεται τυχαία μέσω *RandomWalk*.
- **MountainCar**: τόσο η βαρύτητα (*gravity*) όσο και η δύναμη του κινητήρα (*force*) αυξάνονται σταδιακά μέσω δύο ανεξάρτητων *IncrementUpdate*.

Οι αλγόριθμοι Hill Climbing, Random Restart Hill Climbing και Simulated Annealing εφαρμόζονται online: σε κάθε βήμα ο πράκτορας αξιολογεί τις διαθέσιμες κινήσεις μέσω προσομοίωσης και επιλέγει χωρίς να γνωρίζει τη δυναμική του συστήματος. Οι αλγόριθμοι βασίζονται στο κεφάλαιο 4 του Russell & Norvig.

2 Αλγόριθμοι Τοπικής Αναζήτησης

2.1 Hill Climbing

Ο Hill Climbing (HC) επιλέγει σε κάθε βήμα την κίνηση με τη μεγαλύτερη άμεση αξία, χωρίς να κρατά ιστορικό και χωρίς να κοιτά πέρα από τον επόμενο χρόνο. Είναι ο απλούστερος από τους τρεις αλγορίθμους και λειτουργεί αμιγώς greedy.

Σε κάθε κλήση της *act()*, ο πράκτορας λαμβάνει αντίγραφο του περιβάλλοντος και δοκιμάζει κάθε δυνατή κίνηση σε αυτό. Αν όλες οδηγούν σε τερματισμό, επιστρέφεται -1 και γίνεται τυχαία επιλογή.

Listing 1: Hill Climbing – act(env)

```

1 best_action = -1, best_value = -INF
2 for a in {0, ..., |A|-1}:
3     sim = deepcopy(env)
4     _, r, term, trunc = sim.step(a)
5     if term: continue
6     v = calculate_value(t=sim.env.t, r, term, trunc)
7     if v > best_value:
8         best_value = v; best_action = a
9 return best_action

```

Το κύριο αδύνατο σημείο του HC είναι η εγκλωβίση σε τοπικά βέλτιστα: αν η καλύτερη άμεση κίνηση παραμένει η ίδια, ο αλγόριθμος δεν έχει κανέναν μηχανισμό να βγει από αυτήν, ακόμα και όταν το περιβάλλον έχει αλλάξει σημαντικά.

2.2 Random Restart Hill Climbing

Ο Random Restart Hill Climbing (RRHC) αντιμετωπίζει το πρόβλημα των τοπικών βέλτιστων εκτελώντας N τυχαίες δοκιμές και κρατώντας το καλύτερο αποτέλεσμα. Κάθε δοκιμή επιλέγει τυχαία μια πρώτη κίνηση a_0 , την αξιολογεί σε βάθος H βημάτων και αθροίζει τις αξίες. Χρησιμοποιούμε $N = 20$ και $H = 5$.

Listing 2: Random Restart HC – act(env)

```

1 action_values[a] = -INF for each a
2 for i = 1 to N:
3     sim = deepcopy(env); a0 = random(A)
4     _, r, term, trunc = sim.step(a0)
5     if term: continue
6     total = calculate_value(sim.env.t, r, term, trunc)
7     for j = 2 to H:
8         if term or trunc: break
9         a = random(A)
10        _, r, term, trunc = sim.step(a)
11        total += calculate_value(sim.env.t, r, term, trunc)
12    if total > action_values[a0]:
13        action_values[a0] = total
14 return argmax_a action_values[a]

```

Το πλεονέκτημα έναντι του HC είναι ότι ο ορίζοντας H επιτρέπει στον πράκτορα να αξιολογήσει τις συνέπειες μιας κίνησης πέρα από το άμεσο επόμενο βήμα, κάτι που γίνεται πιο σημαντικό καθώς οι παράμετροι του περιβάλλοντος αποκλίνουν.

2.3 Simulated Annealing

Ο Simulated Annealing (SA) εισάγει πιθανοτική αποδοχή χειρότερων κινήσεων, εμπνευσμένος από τη διαδικασία ανόπτησης μετάλλων. Με υψηλή θερμοκρασία T ο πράκτορας εξερευνά ελεύθερα, ενώ καθώς το T μειώνεται συγκλίνει σε greedy συμπεριφορά.

Αφού αξιολογηθούν όλες οι κινήσεις, η καλύτερη a^* γίνεται αποδεκτή άμεσα. Μια τυχαία εναλλακτική a_c γίνεται αποδεκτή με πιθανότητα $e^{\Delta/T}$, όπου $\Delta = V(a_c) - V(a^*) < 0$.

Listing 3: Simulated Annealing – act(env)

```
1 T = get_temperature(); step_count += 1
2 action_values = {}
3 for a in {0, ..., |A|-1}:
4     sim = deepcopy(env)
5     _, r, term, trunc = sim.step(a)
6     if not term:
7         action_values[a] = calculate_value(sim.env.t, r, term,
8             trunc)
9 if action_values is empty: return -1
10 a_best = argmax action_values
11 a_c = random(action_values.keys())
12 delta = action_values[a_c] - action_values[a_best]
13 if delta >= 0 or U(0,1) < exp(delta / T):
14     return a_c
15 return a_best
```

Στρατηγικές ψύξης. Δοκιμάσαμε τρεις στρατηγικές μείωσης της θερμοκρασίας, όπου n είναι ο αριθμός βημάτων που έχουν εκτελεστεί συνολικά (δεν επαναφέρεται μεταξύ επεισοδίων):

- **Εκθετική (επιλεγμένη):** $T_n = T_0 \cdot \alpha^n$, με $T_0 = 2.0$, $\alpha = 0.95$.
- **Γραμμική:** $T_n = \max(T_0 - \alpha \cdot n, \epsilon)$, $\epsilon = 10^{-3}$.
- **Λογαριθμική:** $T_n = T_0 / \ln(n + 2)$.

Επιλέξαμε την εκθετική γιατί μειώνει την θερμοκρασία ομαλά και δίνει στον αλγόριθμο αρκετό χρόνο εξερεύνησης πριν συγκλίνει. Η λογαριθμική είναι θεωρητικά βέλτιστη αλλά πολύ αργή για πεπερασμένα βήματα, ενώ η γραμμική φτάνει στο κατώτατο όριο ϵ γρήγορα και γίνεται ουσιαστικά ντετερμινιστική.

3 Πειραματική Διαδικασία

Για κάθε περιβάλλον και αλγόριθμο εκτελούμε δύο πειράματα:

Πείραμα 1. Για κάθε τιμή `max_timesteps` $\in \{10, 100, 500, 1000\}$ τρέχουμε 100 επεισόδια και υπολογίζουμε τη μέση αθροιστική ανταμοιβή. Το γράφημα έχει άξονα x το `max_timesteps` και άξονα y τη μέση ανταμοιβή.

Πείραμα 2. Για `max_timesteps` = 100 τρέχουμε 1000 επεισόδια και απεικονίζουμε την ανταμοιβή ανά επεισόδιο, ώστε να φαίνεται πώς επηρεάζει η μεταβολή του περιβάλλοντος την πορεία των αλγορίθμων στον χρόνο.

Η αρχικοποίηση γίνεται με `seed = 42 + ep` για κάθε επεισόδιο `ep` ώστε τα αποτελέσματα να είναι αναπαραγώγιμα. Τα πειράματα τρέχουν χωρίς οπτικοποίηση (`render_mode=None`) από το αρχείο `test/run_experiments.py`.

4 Αποτελέσματα

4.1 CartPole

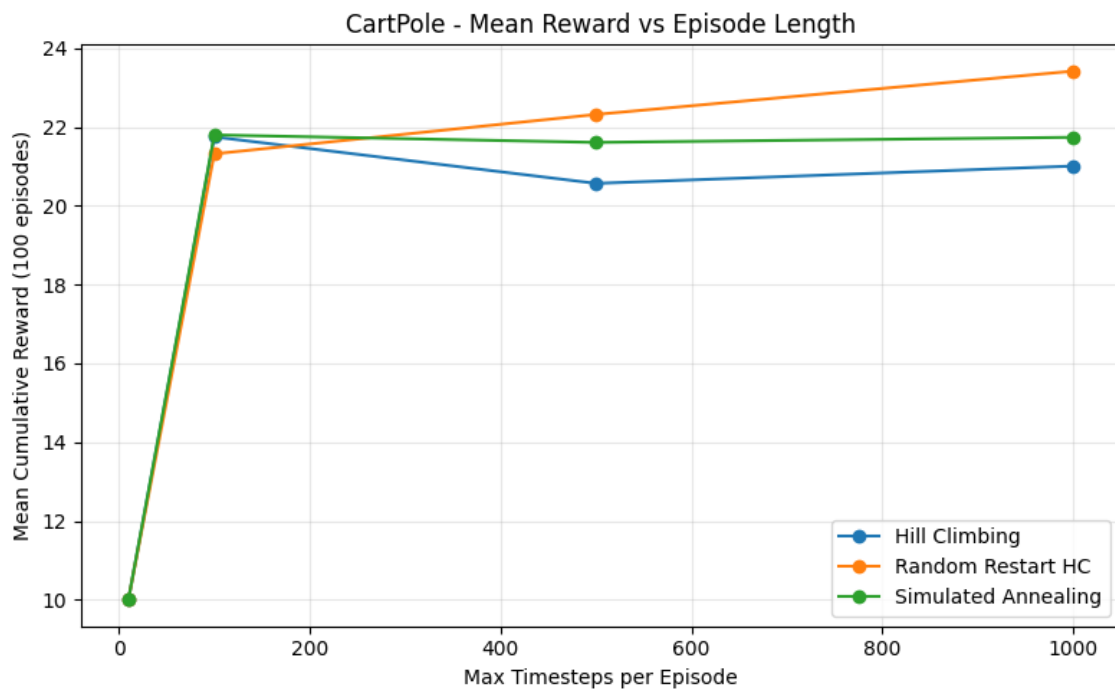


Figure 1: CartPole – Μέση αθροιστική ανταμοιβή ανά `max_timesteps` (100 επεισόδια). Η ανταμοιβή σταθεροποιείται γύρω στα 21–22 βήματα από τα `max_timesteps`=100 και πάνω, αντανακλώντας τον περιορισμό που επιβάλλει η μη-στασιμότητα. Ο RRHC αποδίδει καλύτερα στα μεγαλύτερα `max_timesteps`.

Το Σχήμα 1 αποκαλύπτει ένα ενδιαφέρον μοτίβο: για `max_timesteps= 10` και οι τρεις αλγόριθμοι επιτυγχάνουν τέλεια βαθμολογία 10 (επιβίωση σε όλα τα βήματα), αλλά από `max_timesteps= 100` και άνω η μέση ανταμοιβή παγώνει στα 21–24 βήματα ανεξαρτήτως του διαθέσιμου budget. Αυτό δεν είναι τυχαίο: η σταδιακή αύξηση της `masspole` και η τυχαία μεταβολή της `gravity` κάνουν το περιβάλλον αρκετά δύσκολο ώστε ο πράκτορας να αποτυγχάνει γύρω στο βήμα 21, ανεξάρτητα από το πόσα βήματα επιτρέπονται.

Μεταξύ των αλγορίθμων, ο RRHC ξεχωρίζει στα μεγαλύτερα `max_timesteps`: φτάνει 23.3 στα 1000 βήματα, ενώ ο HC παρουσιάζει ελαφρά πτώση στα 500 (≈ 20.5) πριν ανακάμψει. Ο SA διατηρείται σταθερός στα 21.8 σε όλες τις τιμές, χωρίς αξιοσημείωτη βελτίωση ή επιδείνωση. Το συμπέρασμα είναι ότι ο ορίζοντας $H = 5$ του RRHC του δίνει μικρό πλεονέκτημα όταν υπάρχει αρκετός χρόνος για εξερεύνηση.

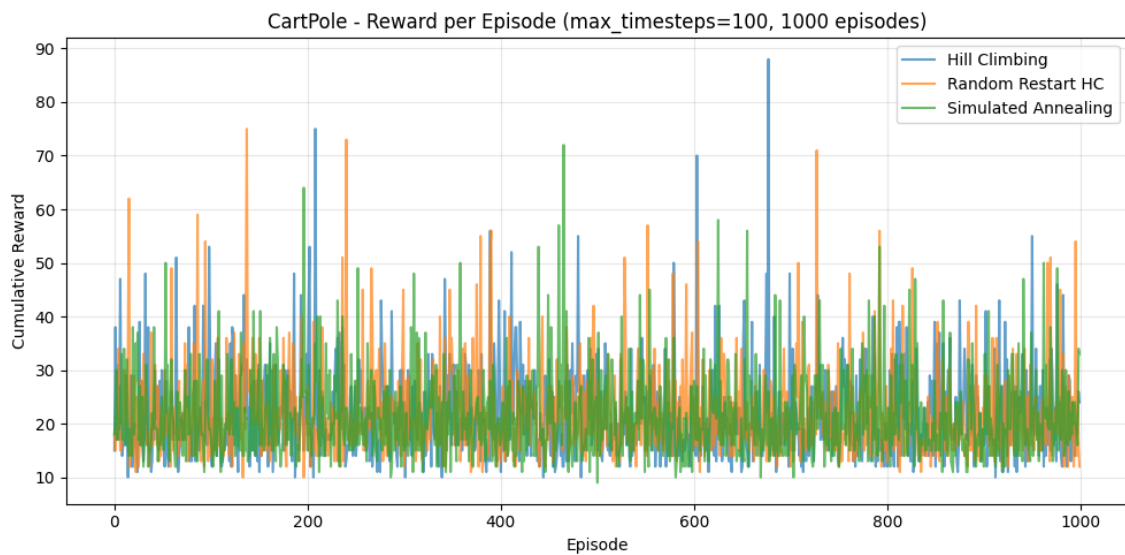


Figure 2: CartPole – Αθροιστική ανταμοιβή ανά επεισόδιο (`max_timesteps=100`, 1000 επεισόδια). Οι τρεις καμπύλες αλληλεπικαλύπτονται σε μεγάλο βαθμό, με υψηλή διακύμανση αλλά χωρίς συστηματική αλλαγή στη συμπεριφορά κατά τη διάρκεια των 1000 επεισοδίων.

Στο Σχήμα 2 οι τρεις αλγόριθμοι παρουσιάζουν παρόμοια και εναλλασσόμενη διακύμανση, με ανταμοιβές που κυμαίνονται από ≈ 10 έως ≈ 88 ανά επεισόδιο. Οι καμπύλες αλληλεπικαλύπτονται έντονα και δεν υπάρχει κάποιος αλγόριθμος που να ξεχωρίζει ξεκάθαρα. Σημαντικό είναι ότι δεν παρατηρείται καμία τάση υποβάθμισης ούτε βελτίωσης κατά τη διάρκεια των 1000 επεισοδίων: οι συνεχείς αλλαγές παραμέτρων επηρεάζουν την ατομική δυσκολία κάθε επεισοδίου αλλά δεν μεταβάλλουν συστηματικά την ικανότητα των αλγορίθμων.

4.2 MountainCar

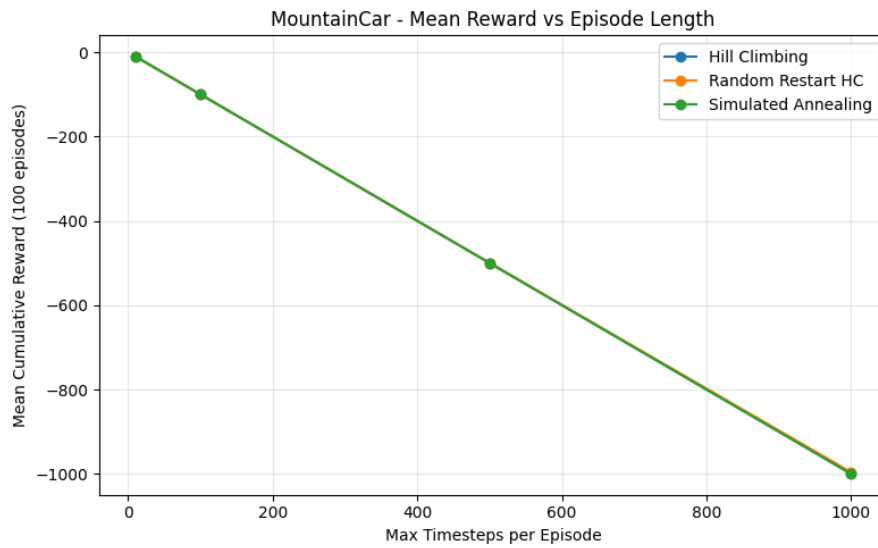


Figure 3: MountainCar – Μέση αθροιστική ανταμοιβή ανά `max_timesteps` (100 επεισόδια). Και οι τρεις αλγόριθμοι ακολουθούν πρακτικά την ίδια γραμμή $y = -\text{max_timesteps}$, δείχνοντας ότι κανένας πράκτορας δεν καταφέρνει να φτάσει στον στόχο.

Οι τρεις καμπύλες του Σχήματος 3 είναι ουσιαστικά αδιαχώριστες: και οι τρεις αλγόριθμοι ακολουθούν γραμμικά την καμπύλη $y = -\text{max_timesteps}$, δηλαδή αθροίζουν -1 ανά βήμα χωρίς ποτέ να επιτύχουν τον στόχο. Η μη-στασιμότητα (αύξηση βαρύτητας και δύναμης) δεν αλλάζει αυτή την εικόνα: το πρόβλημα αντιμετωπίζεται εξίσου ανεπιτυχώς σε όλες τις ρυθμίσεις.

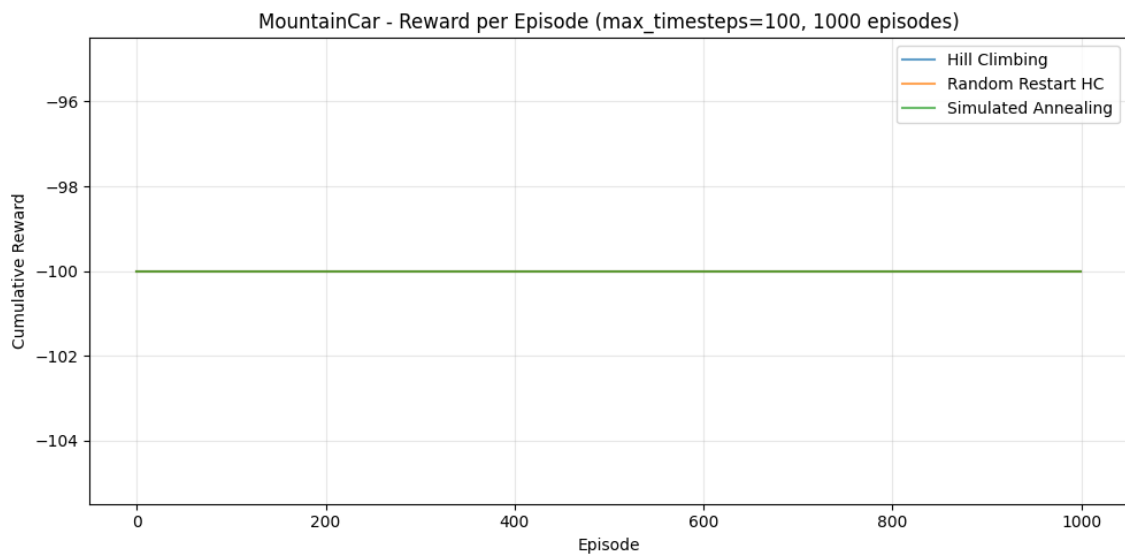


Figure 4: MountainCar – Αθροιστική ανταμοιβή ανά επεισόδιο (`max_timesteps=100`, 1000 επεισόδια). Και οι τρεις αλγόριθμοι παράγουν ακριβώς -100 σε κάθε επεισόδιο, χωρίς καμία εξαίρεση.

Το Σχήμα 4 είναι ίσως το πιο χαρακτηριστικό αποτέλεσμα της εργασίας. Ο άξονας y έχει εύρος μόλις $[-105, -96]$ και οι τρεις καμπύλες εμφανίζονται ως μία και μοναδική οριζόντια γραμμή στο -100 . Σε κανένα από τα 1000 επεισόδια κανένας αλγόριθμος δεν έχει επιτύχει διαφορετικό αποτέλεσμα από το -100 : κάθε επεισόδιο τελειώνει εξαντλώντας όλα τα διαθέσιμα βήματα χωρίς το αυτοκίνητο να αγγίζει ποτέ τον στόχο. Αυτό δείχνει πόσο αδύναμη είναι η one-step lookahead στρατηγική για ένα πρόβλημα που απαιτεί ανάπτυξη ορμής μέσα από πολλά διαδοχικά βήματα.

5 Συμπεράσματα

Στο CartPole, και οι τρεις αλγόριθμοι αντεπεξήλθαν ικανοποιητικά, επιβιώνοντας κατά μέσο όρο γύρω στα 21–24 βήματα ανά επεισόδιο. Ο HC αποδείχθηκε ανταγωνιστικός παρά την απλότητά του, αφού η εξισορρόπηση ράβδου είναι εκ φύσεως αντιδραστική εργασία και η άπληστη επιλογή ενός βήματος αρκεί. Ο RRHC πέτυχε την καλύτερη μέση βαθμολογία στα μεγαλύτερα `max_timesteps` (≈ 23.3 στα 1000 βήματα), καθώς ο ορίζοντας $H = 5$ επιτρέπει μια πιο ολοκληρωμένη αξιολόγηση κάθε κίνησης. Ο SA παρέμεινε σταθερός αλλά δεν κατόρθωσε να ξεπεράσει τους άλλους δύο: η στοχαστική αποδοχή χειρότερων κινήσεων δεν προσθέτει αξία σε ένα περιβάλλον όπου τα τοπικά βέλτιστα δεν είναι το κεντρικό πρόβλημα. Αξιοσημείωτο είναι επίσης ότι η απόδοση όλων των αλγορίθμων δεν υποβαθμίζεται κατά τα 1000 επεισόδια, πράγμα που δείχνει ότι το άμεσο σήμα ανταμοιβής του CartPole είναι αρκετά πλούσιο ώστε να στηρίζει σωστές αποφάσεις ακόμα και όταν οι παράμετροι του περιβάλλοντος έχουν αλλάξει.

Στο MountainCar το αποτέλεσμα είναι σαφές: και οι τρεις αλγόριθμοι αποτυγχάνουν πλήρως. Η ανταμοιβή είναι ακριβώς -100 σε κάθε ένα από τα 1000 επεισόδια, χωρίς καμία εξαίρεση. Το πρόβλημα δεν είναι οι αλγόριθμοι – είναι η θεμελιώδης αδυναμία της one-step lookahead στρατηγικής για μια εργασία που απαιτεί ανάπτυξη ορμής μέσα από συντονισμένες, πολλαπλές κινήσεις. Το άμεσο σήμα ανταμοιβής (-1 σε κάθε βήμα ανεξάρτητα από τη θέση) δεν δίνει καμία πληροφορία που να βοηθήσει έναν greedy πράκτορα να διαφοροποιήσει καλές από κακές κινήσεις. Για να λυθεί το MountainCar με τοπική αναζήτηση θα χρειαζόταν είτε πολύ μεγαλύτερος ορίζοντας στον RRHC (πολλαπλασιασμός του H) είτε διαφορετική συνάρτηση ανταμοιβής που να ανταμείβει ανάλογα με την απόσταση από τον στόχο.